



MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF COMPUTER SYSTEMS ENGINEERING

DIP Project Report

Creating a Vehicle Number Plate Detector

Names: Mashaal Zulfikar, Junaid Chandio, Aman Khuwaja

Roll No.: 21CS105, 21CS115, 19-21CS045

Class: **21CS-1**

Date: **13-11-2024**

Submitted to: **Dr. Ahsan Ansari**

DIP Project Report - Creating a Vehicle Number Plate Detector in MATLAB

Group Members:

M. Mashaal Zulfikar	Junaaid Chandio	Aman Khuwaja
21CS105	21CS115	19-21CS045

Introduction:

In this project, we have made a program with the help of MATLAB that detects and recognizes vehicles plate numbers. As the number of vehicles on roads continues to rise, manually enforcing traffic laws and regulations for smooth traffic flow is becoming increasingly challenging. Traffic management systems needs room for improvement to identify and monitor vehicles by their license plate number to make vehicle identification essential for automation and efficiency.

In each country, vehicles have a unique license number on their license plate, allowing for clear identification, even between vehicles of the same make and model. Over recent years, various algorithms have emerged to create a system for license plate detection, each with its own strengths and limitations. This project details a method that identifies license plates through Digital Image Processing course topics, such as Image Acquisition, Pre-processing, Image Segmentation, Edge Detection, Binarization, Morphological Operations, and Character Recognition.. To improve accuracy in the process:

- post-processing merges multiple detections of the same plate, while trackers narrow the search to specific image regions.
- The approach uses binarization to differentiate characters from the plate's background, creating black-and-white regions from which non-essential areas are eliminated, leaving the region most likely containing the plate number.

Topics Concept Linked to this project:

For detection and recognition of number plates on car samples, we have taken help and applied concepts of the following topics from our Digital Image Processing course to our project:

1. **Image Acquisition:** Capturing images or frames that contain license plates.
2. **Pre-processing:** Enhancing image quality, like noise reduction and contrast adjustment, to prepare for further processing.
3. **Image Segmentation:** Isolating the region of interest (the license plate) from the background.
4. **Edge Detection:** Identifying the boundaries of the license plate and characters.
5. **Binarization:** Converting the image to black and white based on contrast to highlight the characters.
6. **Morphological Operations:** Refining regions and shapes, useful for character segmentation.
7. **Character Recognition:** Using Optical Character Recognition (OCR) techniques to read the alphanumeric characters.

Purpose of this project:

The main purpose of this project is to detect a license plate from an image provided to **MATLAB R2013**. An efficient algorithm is developed to detect a license plate in various luminance conditions. This algorithm extracts and processes license plate data from the image of car for car license plate identification/recognition.

Tools Used: MATLAB R2013a and sample images of cars with a plate number.

Digital Image Processing Concepts:

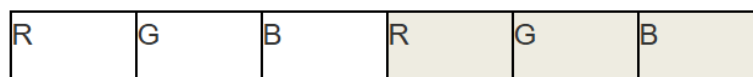
In digital image processing, understanding the structure and composition of an image is fundamental to developing an automated license plate recognition system. An image can be seen as an arrangement of tiny, data-storing elements called pixels on a two-dimensional plane. Each pixel encodes information about the image's properties, such as light intensity, color, and luminance. These properties are essential for capturing and interpreting the details necessary for detecting a license plate.

Every pixel has a specific format, commonly RGB or YCbCr, that determines how its information is stored and processed. These formats help capture the contrast between the license plate characters and the background, making image processing tasks like segmentation and character recognition possible. Pixel values create the basis for different image types: RGB images show full color, grayscale images capture brightness levels, binary images separate the foreground from the background, and indexed images use color mapping to save memory.

Pixel Format:

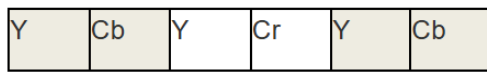
Two primary pixel formats used in image processing for applications like license plate recognition are RGB and YCbCr:

- **RGB Format:** In the RGB format, each pixel is defined by three color components: Red, Green, and Blue. These three components combine to produce the final color of the pixel, with each component value ranging from 0 to 255. For instance, (0, 0, 0) represents black, (255, 0, 0) represents red, and (0, 255, 0) represents green. Each component typically uses 8 bits of storage, making RGB images memory-intensive but very effective for capturing color details. RGB can be particularly useful when identifying the color aspects of a license plate or recognizing color-based distinctions.



Representation of pixels in RGB Format

- **YCbCr Format:** The YCbCr format separates the image's luminance (Y) from chrominance (Cb and Cr). Luminance captures light intensity, while Cb and Cr capture the color information. This separation allows for more efficient storage and processing, as YCbCr images can often provide enough detail with less memory usage than RGB. The 4:2:2 YCbCr format, for example, interleaves the chrominance components, allowing alternate pixels to share color information without significant loss in quality—ideal for high-speed, real-time applications such as automatic number plate recognition.



Representation of pixels in YCbCr Format



Luminosity (Y) + Chroma (Cb, Cr) → YCbCr

Image Types:

In digital image processing, various image types offer different ways of handling data, allowing for optimized processing depending on the requirements of the automated number plate recognition system.

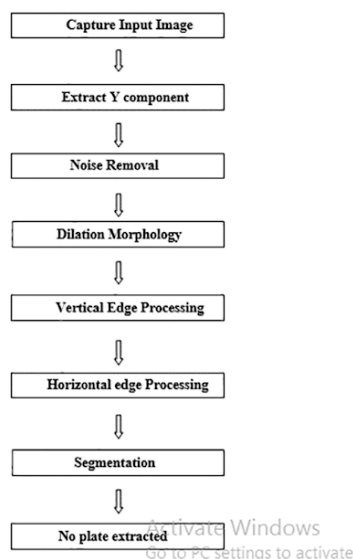
- **RGB Images:** RGB images are “true color” images where each pixel is defined by the intensities of the red, green, and blue color components. Combining different values of these primary colors allows the RGB format to represent nearly all visible colors. This format is essential for applications needing precise color data, such as distinguishing between characters and backgrounds on license plates that may have specific color contrasts. RGB images provide a rich data source, but they also require more storage and processing time.
- **Intensity Images (Grayscale Images):** Intensity images, or grayscale images, represent the pixel's brightness using shades of gray, where each pixel contains a single intensity value rather than three RGB components. By reducing an image to a grayscale format, the system can focus on the intensity contrasts rather than color, which often simplifies the process of detecting characters against a license plate's background. Grayscale is particularly useful for edge detection, binarization, and other preliminary steps in license plate recognition, as it reduces data without compromising important contrast information.
- **Binary Images:** Binary images consist of pixels that have only two possible intensity values, usually black (0) and white (1 or 255). They are commonly produced by thresholding grayscale images to create a clear distinction between foreground (license plate characters) and background. Binary images are useful in license plate recognition as they simplify processing by reducing each pixel to a binary state, allowing for straightforward detection of character regions. This can be particularly helpful in segmenting characters for further recognition.

- **Indexed Images:** Indexed images use a data matrix (X) and a colormap (map) matrix, where each pixel is assigned a value that corresponds to an entry in the colormap. This format can be more memory-efficient, as colors are mapped instead of stored per pixel. Indexed images are commonly used in applications where color information is necessary but needs to be handled in a compact way. This efficiency can be advantageous in systems processing large datasets or multiple frames, as it minimizes memory use without sacrificing color accuracy.

Analyzing Algorithm for Number Plate Detection with MATLAB Implementation:

The license plate detection algorithm is implemented in MATLAB due to its powerful capabilities in image processing, extensive library functions, and user-friendly environment. MATLAB's efficient computational framework allows for quick testing and debugging of algorithms, making it ideal for preliminary development and validation before deploying on physical hardware. This approach mitigates potential errors that could arise when directly implementing on hardware, which is often more complex to troubleshoot.

Flowchart of Algorithm:



- **Capture Input Image**

Capture or import the vehicle image in RGB format as the primary input.

- **Extract Y Component**

Extract the brightness (Y) component to focus on intensity contrasts rather than color.

- **Noise Removal**

Apply filters to remove noise, improving edge clarity and overall image quality.

- **Dilation Morphology**

Use dilation to enhance edges, fill gaps, and make the license plate and characters more distinguishable.

- **Vertical Edge Processing**

Analyze vertical intensity changes to detect column-wise features around license plate text.

- **Horizontal Edge Processing**

Examine horizontal intensity shifts to enhance row-wise edges of the license plate.

- **Segmentation**

Isolate regions likely to contain the license plate by segmenting the image based on edge detection.

- **Plate Extraction**

Extract the identified region with high probability of containing the license plate; if not found, it outputs "No plate extracted."

Algorithm Steps:

1. Converting to Grayscale

The detection algorithm relies on the gray levels of an image, making it necessary to convert colored images to grayscale. This step removes color variations, focusing on brightness differences, which are crucial for detecting license plate edges and shapes.

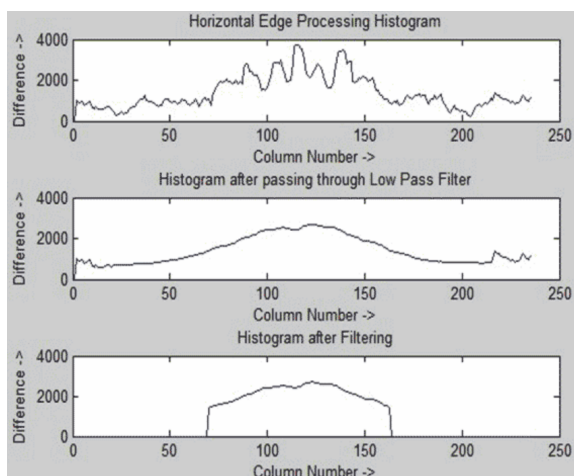
2. Image Dilation

Dilation enhances edges, fills gaps, and removes noise, improving edge clarity in grayscale images. This step is essential for license plate detection, as it sharpens boundaries and increases brightness contrast, helping highlight the license plate area.



3. Horizontal and Vertical Edge Processing

Using histograms of gray-level differences, the algorithm calculates the sum of differences for neighboring pixels in each row and column, capturing edge transitions that are common around text and plate boundaries. This provides a structured map for locating probable plate regions.



4. Low-Pass Filtering on Histograms

To reduce drastic histogram value changes and preserve important information, low-pass filtering smooths out variations by averaging nearby values. This adjustment ensures that high-value areas, which likely contain the plate, remain clear and detectable.

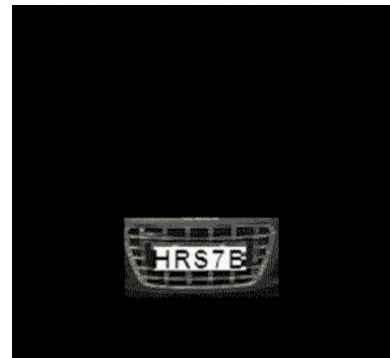
5. Filtering Unwanted Regions

The filtered histograms indicate regions with high contrast, typical of license plates with characters. Areas with low histogram values are removed by applying a dynamic threshold, isolating regions with high likelihoods of containing a license plate.



6. Segmentation and Region of Interest Extraction →

High-value histogram regions are identified as potential license plate zones. The final algorithm step isolates the region with maximum histogram values as the most probable license plate, which is then extracted for analysis.



MATLAB Source Code:

```
clc; % Clear command window.
clear all; % Delete all variables.
close all; % Close all figure windows except those created by
imtool close all; % Close all figure windows created by imtoo
workspace; % Make sure the workspace panel is showing.

% Read Image
I = imread ('CAR2.jpg');
figure(1);
imshow(I);

% Extract Y component (Convert an Image to Gray)
Igray = rgb2gray(I);
[rows cols] = size(Igray);
```

```

%% Dilate and Erode Image in order to remove noise
Idilate = Igray;
for i = 1:rows
    for j = 2:cols-1
        temp = max(Igray(i,j-1), Igray(i,j));
        Idilate(i,j) = max(temp, Igray(i,j+1));
    end
end

I = Idilate;
figure(2);
imshow(Igray);
figure(3);
title('Dilated Image')
imshow(Idilate);
figure(4);
imshow(I);
difference = 0;
sum = 0;
total_sum = 0;
difference = uint32(difference);

%% PROCESS EDGES IN HORIZONTAL DIRECTION
disp('Processing Edges Horizontally...');
max_horz = 0;
maximum = 0;
for i = 2:cols
    sum = 0;
    for j = 2:rows
        if(I(j, i) > I(j-1, i))
            difference = uint32(I(j, i) - I(j-1, i));
        else
            difference = uint32(I(j-1, i) - I(j, i));
        end

        if(difference > 20)
            sum = sum + difference;
        end
    end
end

```

```

end

horz1(i) = sum;

% Find Peak Value
if(sum > maximum)
    max_horz = i;
    maximum = sum;
end

total_sum = total_sum + sum;
end

average = total_sum / cols;
figure(5);
% Plot the Histogram for analysis
subplot(3,1,1);
plot (horz1);
title('Horizontal Edge Processing Histogram');
xlabel('Column Number ->');
ylabel('Difference ->');

%% Smoothen the Horizontal Histogram by applying Low Pass Filter
sum = 0;
horz = horz1;
for i = 21:(cols-21)
    sum = 0;
    for j = (i-20):(i+20)
        sum = sum + horz1(j);
    end

    horz(i) = sum / 41;
end

subplot(3,1,2);
plot (horz);
title('Histogram after passing through Low Pass Filter');
xlabel('Column Number ->');

```

```

ylabel('Difference ->');

%% Filter out Horizontal Histogram Values by applying Dynamic
disp('Filter out Horizontal Histogram...');
for i = 1:cols
    if(horz(i) < average)
        horz(i) = 0;
        for j = 1:rows
            I(j, i) = 0;
        end
    end
end

subplot(3,1,3);
plot (horz);
title('Histogram after Filtering');
xlabel('Column Number ->');
ylabel('Difference ->');

%% PROCESS EDGES IN VERTICAL DIRECTION
difference = 0;
total_sum = 0;
difference = uint32(difference);
disp('Processing Edges Vertically...');
maximum = 0;
max_vert = 0;

for i = 2:rows
    sum = 0;
    for j = 2:cols
        if (I(i, j) > I(i, j-1))
            difference = uint32(I(i, j) - I(i, j-1));
        end

        if (I(i, j) <= I(i, j-1))
            difference = uint32(I(i, j-1) - I(i, j));
        end
    end
end

```

```

        if (difference > 20)
            sum = sum + difference;
        end
    end

    vert1(i) = sum;

    % Find Peak in Vertical Histogram
    if (sum > maximum)
        max_vert = i;
        maximum = sum;
    end

    total_sum = total_sum + sum;
end

average = total_sum / rows;

figure(6);
subplot(3,1,1);
plot(vert1);
title('Vertical Edge Processing Histogram');
xlabel('Row Number ->');
ylabel('Difference ->');

%% Smoothen the Vertical Histogram by applying Low Pass Filter
disp('Passing Vertical Histogram through Low Pass Filter...')
sum = 0;
vert = vert1;
for i = 21:(rows-21)
    sum = 0;
    for j = (i-20):(i+20)
        sum = sum + vert1(j);
    end
    vert(i) = sum / 41;
end

subplot(3,1,2);

```

```

plot(vert);
title('Histogram after passing through Low Pass Filter');
xlabel('Row Number ->');
ylabel('Difference ->');

%% Filter out Vertical Histogram Values by applying Dynamic T
disp('Filter out Vertical Histogram...');
for i = 1:rows
    if (vert(i) < average)
        vert(i) = 0;
        for j = 1:cols
            I(i, j) = 0;
        end
    end
end

subplot(3,1,3);
plot(vert);
title('Histogram after Filtering');
xlabel('Row Number ->');
ylabel('Difference ->');
figure(7), imshow(I);

%% Find Probable Candidates for Number Plate
j = 1;
for i = 2:cols-2
    if (horz(i) ~= 0 && horz(i-1) == 0 && horz(i+1) == 0)
        column(j) = i;
        column(j+1) = i;
        j = j + 2;
    elseif ((horz(i) ~= 0 && horz(i-1) == 0) || (horz(i) ~= 0
        column(j) = i;
        j = j + 1;
    end
end

j = 1;
for i = 2:rows-2

```

```

        if (vert(i) ~= 0 && vert(i-1) == 0 && vert(i+1) == 0)
            row(j) = i;
            row(j+1) = i;
            j = j + 2;
        elseif ((vert(i) ~= 0 && vert(i-1) == 0) || (vert(i) ~= 0
            row(j) = i;
            j = j + 1;
        end
    end

    [temp, column_size] = size(column);
    if (mod(column_size, 2))
        column(column_size+1) = cols;
    end

    [temp, row_size] = size(row);
    if (mod(row_size, 2))
        row(row_size+1) = rows;
    end

    %% Region of Interest Extraction
    % Check each probable candidate

    for i = 1:2:row_size
        for j = 1:2:column_size
            % If it is not the most probable region, remove it from
            if ~(max_horz >= column(j) && max_horz <= column(j+1))
                % This loop is only for displaying proper output
                for m = row(i):row(i+1)
                    for n = column(j):column(j+1)
                        I(m, n) = 0;
                    end
                end
            end
        end
    end
end
end
end

```

```
figure(8), imshow(I);  
imshow(I);
```

Conclusion & Future Implementation:

This project demonstrates the successful detection and extraction of vehicle number plates. Through grayscale conversion, noise reduction, and edge processing, we identified and isolated the probable region of the number plate and accurately extracted the text (e.g., "HRS7B"). Horizontal and vertical edge processing helped pinpoint specific columns, allowing us to narrow down the region of interest for text extraction.

For future improvements, the system could be enhanced to recognize text in various formats, helping to detect plates that deviate from standard formats, which would improve security by identifying attempts to mislead the system. Additionally, integrating the system with an internet-connected database would enable real-time updates of criminal records, allowing data to be shared instantly with other stations.